01

Building Agents

Day 1 | Agentic AI fundamentals, and where an agent belongs in a workflow

Abhimanyu S.

2 hours · Day 1 of 4 · three live builds

abhimanyu@novnex.com

The programme

Four sessions, one thread: from one working agent to a governed fleet.

DAY 1

Fundamentals

What an agent is, and where it belongs in a workflow

YOU LEAVE ABLE TO

Build one from nothing, and read any other.

YOU ARE HERE

DAY 2

Processes

Steps, triggers, and a low-touch process that runs without you

YOU LEAVE ABLE TO

Design a process an agent can safely run.

DAY 3

Testing

Agents that write the tests, catch a contract break, and triage failures

YOU LEAVE ABLE TO

Wire three agents into one pipeline.

DAY 4

The Ecosystem

Inventory, identity, run records and cost

YOU LEAVE ABLE TO

Run agents beyond the team that built them.

What gets built in one session is still running in the next. Nothing here is throwaway.

What today is

A short piece of theory, then three live builds - the same agent, taken further each time.

THE FRAME

- Every agent is four parts: reasoning, memory, tools, planning
- Under the hood it is one loop, re-run until something stops it
- Every action it takes is a tool call that ordinary code runs
- You configure five fields. The four parts are what they become

THE BUILDS

- One product today: GitHub Copilot in VS Code
- One agent, built from nothing and carried through all three
- Live, with nothing prepared except the repository it works on
- Every build ends by naming exactly what changed - one thing

By the close, one agent will have gone from an empty file to a working reviewer - and you will have seen every change that got it there.

Today at a glance

Three of the seven blocks are a live screen.

TALK	**How an agent works**	Four parts, one loop - and the five fields you fill in	15
BUILD	**An agent from nothing**	Create it, ask it, ground it, ask it again	20
TALK	**Instructions and grounding**	Put a fact in the right place the first time	10
BUILD	**Tools: the wrong one, then the right one**	Two tools, one question, three strings	20
TALK	**Where an agent belongs in a process**	Mark a step agent, machinery or human - and defend it	10
BUILD	**The same agent, somewhere else**	The terminal: same files, second surface, zero rebuild	20
Q&A	**Questions**	Bring the ones the builds raised	10

1

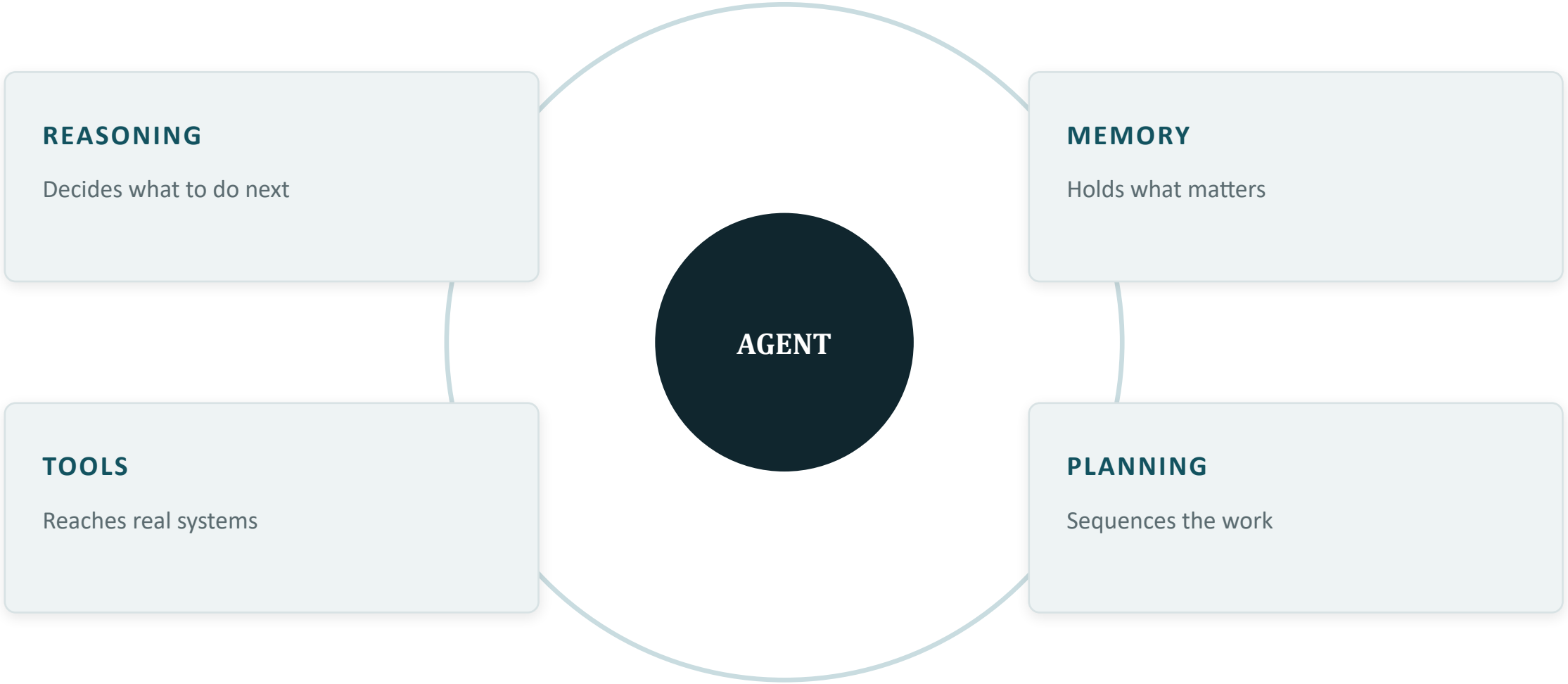
How an agent works

Four parts, one loop - and the five fields you actually fill in

15 MINUTES

Every agent has four parts

Different products wire them differently. All four are always present.



When an agent misbehaves, the diagnostic question is not "is the AI bad" - it is "which of these four failed".

What people imagine

An intuitive picture — and the reason you have it

THE IMAGINED AGENT

- thinking
- deciding
- reaching out
- acting on its own

A persistent thing that sits there and does things, whether or not anyone is watching.

WHAT IS ACTUALLY THERE

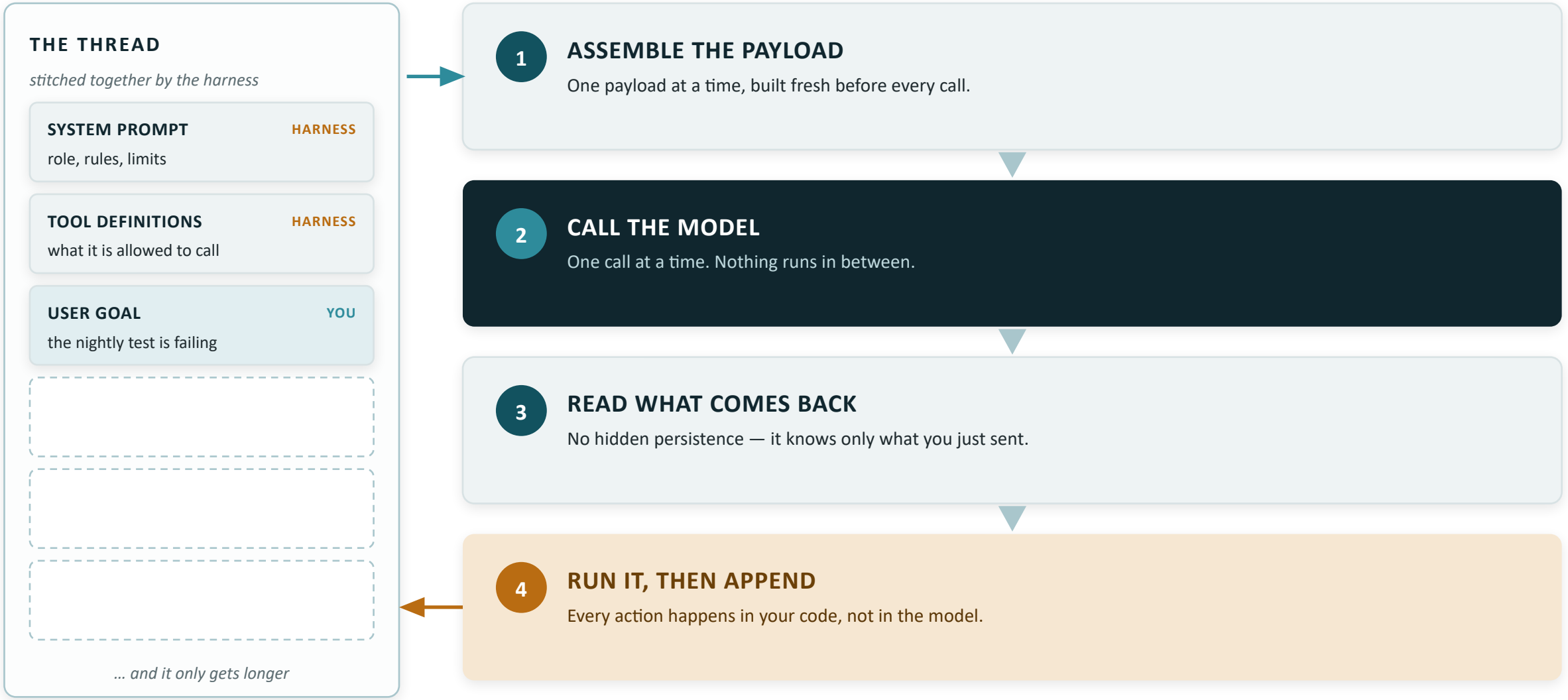
Between two calls, nothing exists at all.

Nothing is waiting. Nothing is thinking. Nothing is running between one call and the next.

The sense of a continuous “it” is manufactured — the harness re-sends the whole history every time, so the thing appears to have been there all along.

What is actually happening

A thread the harness maintains, and a loop that keeps re-reading it



Step 1 — your code assembles one payload

Three things go in, joined into one body of text — every single call

Step 2 — the payload is sent as one request

It remembers nothing between calls. Continuity is something you re-send.

Working memory is the payload

It is whatever fits in the payload — and the payload has a ceiling

Step 3 — the model returns text

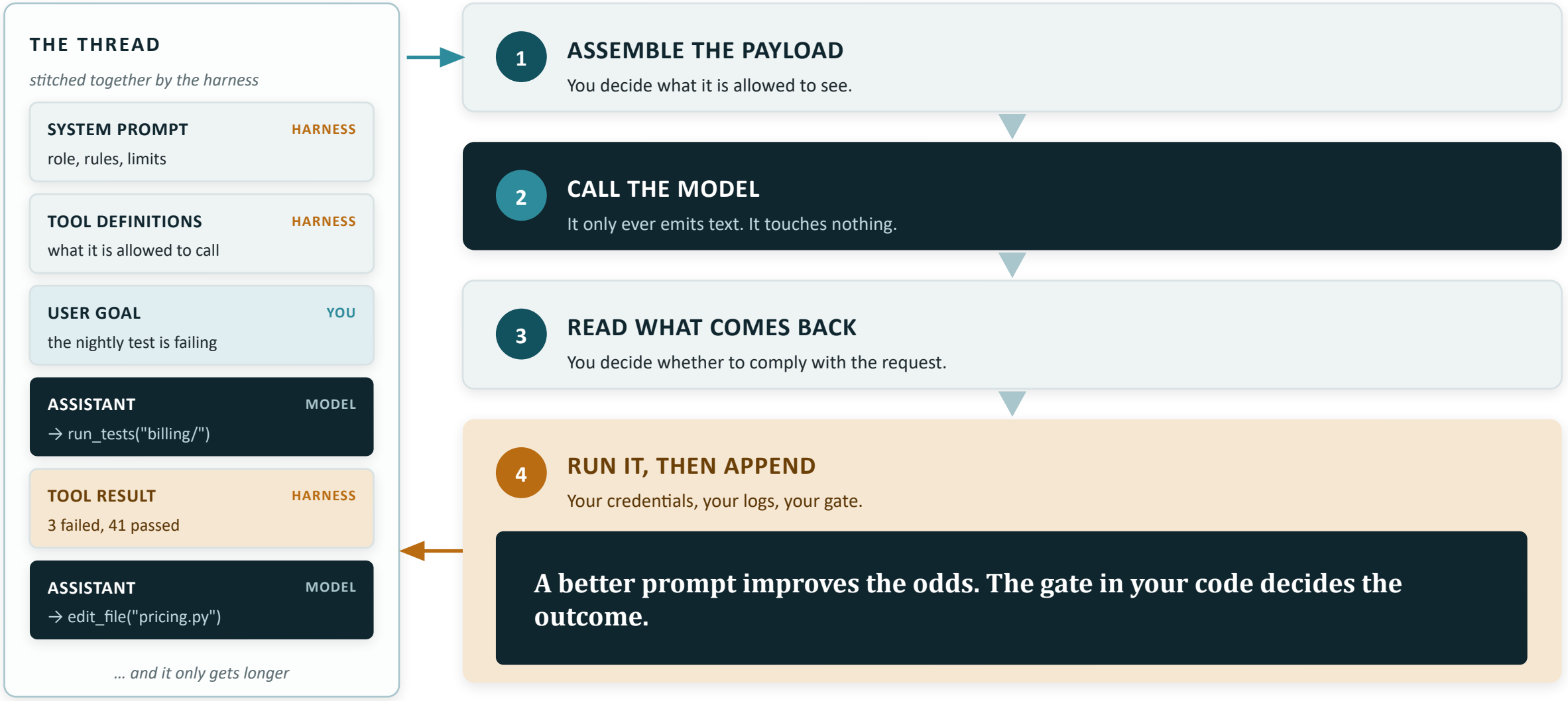
A final answer, or a request for a tool. It cannot run anything itself.

Step 4 — your code runs the tool and appends the result

Your credentials, your permissions, your logs — then round it goes

Where control really lives

Two layers — one of them asks, the other decides



The same cycle, in plain language

This is the vocabulary people use for what you just saw under the hood

Cost, latency and risk all scale with the number of times this loop runs. That single sentence answers most of the finance questions you will get.

The five fields you fill in

Every build screen you will ever see asks for these five. Each one feeds one of the four parts.

REASONING	**Instructions** How it judges, what it refuses, the shape of the answer.	`"Never call a story ready if an acceptance criterion cannot be tested."`
MEMORY	**Grounding** The facts it may treat as true - retrieved into the payload each run.	`standards/definition-of-ready.md`
TOOLS	**Tools** What it can call, and therefore what it can change.	`get_component_owner(service)`
PLANNING	**Trigger + Stop** Planning is the loop. You write its two ends - what starts it, what ends it.	`starts: a person, a schedule, an event · ends: drafted, not posted`

When an agent misbehaves: name the part, then fix its field. Only reasoning’s field is a nudge rather than a decision - and that is where everybody reaches first.

The five fields, in GitHub Copilot

One markdown file in the repository, plus the files it is allowed to read.

1

INSTRUCTIONS

`.github/agents/`

`story-check.agent.md`

Reviewed like any other change.

2

GROUNDING

`standards/*.md`

`.github/instructions/*.instructions.md`

No applyTo line, never applied.

3

TOOLS

`tools: [ ... ]`

`plus the Configure Tools picker`

The ceiling is 128 tools per request.

4

TRIGGER

`you, in the chat box`

`or an issue, or a schedule`

Still a person, most days.

5

STOP

`the approval dialog`

`once · session · workspace · always`

The fourth one is standing.

The whole agent is files. Changing any of these is a pull request - reviewed, versioned, and revertible on a bad Monday.

2

Build - an agent from nothing

Create it, ask it, ground it, ask it again

LIVE · 20 MINUTES

Build 1 - the setup

Story Check: it reads a user story and says whether it meets our Definition of Ready.

WHAT WE ARE BUILDING

A user story arrives. Is it ready to be picked up, and if not, what is missing?

- The standard is already in the repository
- The agent starts with no idea it is there
- Trigger: me, typing in the chat

```
Is STORY-4471 ready to pick up?
```

FOUR THINGS TO WATCH

1. **It exists after one sentence.**
   One description in, and the product writes the agent file for itself. The file is the agent.

2. **The first answer is generic - and confident.**
   Our standard is already in the repository, and nothing will tell you it never looked at it.

3. **One line changes. Nothing else.**
   Not the model, not the question, not the standard.

4. **The chat shows what it read.**
   Every file it opened is listed on the reply. What is missing from that list is the finding.

The whole build is one file, and the fix is one line. Watch how far the answer moves.

Build 1 - what just happened

One line changed. Everything people usually reach for did not.

WHAT CHANGED

One line of instructions

"Judge only against standards/definition-of-ready.md, and quote the criterion number for every finding." One sentence, pointing at a file that was already there.

Cost: nothing. Skill required: none.

WHAT DID NOT CHANGE

Everything else

The model is the same. The question I typed was word for word the same. The standard sat unchanged in the repository through both runs. Nobody wrote a better prompt.

So the improvement was not intelligence.

WHAT THE LOG SHOWED

Two reference lists

First run: it read the story, and nothing else. Second run: the story and the standard. The gap between those two lists is the entire diagnosis.

Read the list nobody reads.

The standard was present the whole time. Presence is not grounding - grounding is what the run actually read.

3

Instructions and grounding

What actually changes the answer - and what only looks like it should

10 MINUTES

Instructions or grounding?

Almost every weak agent I have looked at has a fact in the wrong box.

INSTRUCTIONS · BEHAVIOUR

Scope. Tone. The shape of the answer. What to refuse. What order to work in.

```
"Answer in three sections: verdict, what is missing, suggested wording."
```

Changes when the reader changes.

GROUNDING · FACTS

The standard. The catalogue. Last quarter's decisions. Anything somebody else owns.

```
"A story is ready when it has a testable acceptance criterion per behaviour."
```

Changes when the world changes.

The test: if it would change when the policy changes, it is grounding. It belongs in a file the agent reads, not in the agent file.

The common failure

Pasting the whole standard into the agent file. It costs you: the entire text re-sent with every single call, no way to tell which part the answer leaned on - and a policy change becomes an agent change, with whatever review that needs in your organisation.

What an instruction will and will not do

The first two on the right are straight from the documentation. The other two are how the machinery works.

IT WILL

- Narrow the scope, and refuse what falls outside it
- Fix the shape of the answer - length, sections, fields
- Name a tool, and say when to reach for it
- Set the order of work when there are several steps

IT WILL NOT

- Auto-apply a path rule whose applyTo line is missing - ever
- Guarantee which of two matching files wins - none is promised
- Guarantee a tool is chosen - naming it raises the odds
- Stop a granted tool - the approval dialog does that

HOW TO WRITE THEM SO THEY WORK

Short, self-contained statements - one rule per line. Scope path rules with applyTo instead of piling everything into one file.

Name tools by their exact spelling - a near-miss hurts selection. And never write two files that disagree; no order is promised.

An instruction is a request the model usually honours. The gate that cannot be talked past is the approval dialog - and we will meet it.

Grounding is files - and files fail politely

Four ways the grounding you set up stops working, none of which shows an error.

STALE

A copy in the repo does not follow the original.

The policy owner edits the SharePoint version. Your repo copy stays as it was, and the agent quotes it with total confidence. Decide which copy is the source of truth, and who refreshes it.

SILENT

A path rule without applyTo is never auto-applied.

The file sits in the repository, correct and committed, doing nothing. No error, no warning, nothing changes colour. You find out because the answers got quietly worse.

IN REACH

Everything in the workspace is grounding, meant or not.

Old fixtures, an abandoned draft of the standard, a stray .env - the agent can read whatever its file tools reach. What is in the folder is in the agent.

NO ORDER

Two matching files are combined, in no promised order.

Write two rules that disagree and you have built a coin flip. Keep one file per concern, and never encode the same rule twice.

After you change grounding, ask a question you already know the answer to. If the answer did not move, the grounding is not live.

4

Build - giving it tools

Two tools, one question, and the three strings that decide the choice

LIVE · 20 MINUTES

Build 2 - the setup

Two tools go on the same agent. Both are plausible. Only one answers the question.

ONE SMALL SERVER, TWO TOOLS

`search_backlog`

`"Searches items."`

`search_owners`

`"Searches items and returns information."`

Forty lines of Python, speaking MCP - the open protocol that exposes a system's functions to an agent.

```
Is STORY-4471 ready to pick up, and which
team would own the work?
```

FOUR THINGS TO WATCH

1. **The trust prompt comes first.**
   A server is code. The editor asks once, by name, before it will run it. That is a gate, not friction.
2. **It may choose wrong. It may choose right.**
   Selection is not deterministic. Same question, two runs, two routes.
3. **Three strings change, in order of power.**
   The tool name. Then the description. Then one line of instructions naming the tool.
4. **The model does not change.**
   Nobody switches to a bigger model, and nobody writes a longer prompt.

If it picks the right tool first time, that is not the demo failing. Ask how you could have known - then break it on purpose.

Build 2 - what just happened

Three strings, in that order. Not a bigger model, and not a longer prompt.

WHY IT CHOSE WRONG

- The selector sees two things: the name, and one line of description. The code behind them it never sees
- Two tools whose names both begin "search" are one tool as far as selection is concerned
- The descriptions described nothing - neither said when to reach for it

THREE FIXES, STRONGEST FIRST

1. **Name it for what it returns.**
   get_component_owner beats search_owners - the name is most of the signal.
2. **Write the description for the selector.**
   Say when to use it and when not to. It is read by the thing choosing, not by a colleague.
3. **Name the tool in the instructions.**
   Exactly as spelled, and say at what point in the work it should be called.

WHAT YOU DO NOT DO

Add a third tool. Lengthen the prompt. Change the model. All three are the reflex, and none of them is the fix.

Tool selection is a naming problem before it is an intelligence problem.

The two fields nobody fills in

Everything built today was started by a person and stopped by running out of things to say.

TRIGGER · WHAT CAN START A RUN

- A person, in the chat box - every run you saw today
- An issue assigned to the cloud agent on GitHub - a pull request comes back
- A schedule wrapped around the terminal - the same agent, run by cron or CI

The unattended versions need things the chat box does not: a repository host, a paid plan, and an owner. That is a decision, not a checkbox.

STOP · WHERE THE RUN ENDS

- The dialog: allow once · this session · this workspace · always
- Settings that pre-approve commands, with allow and deny lists
- And the instruction "stop after writing" - which is a request

"Always" is a standing grant in a settings file, written by one person, at speed. It can be removed. Nothing will ever remind anybody it is there.

Filling in the trigger costs a licensing conversation. Filling in the stop is free - and it is the one people skip.

5

Where an agent belongs

Which step of a process is agent work, which is machinery, and which is neither

10 MINUTES

One process, six steps

A story arrives and has to reach Ready. Mark every step before you build anything.

AGENT	**Read the story and the standard together**	*Messy input, no rule survives it*
AGENT	**Judge whether each acceptance criterion is testable**	*This is the whole reason the agent exists*
MACHINERY	**Look up the component owner**	*One input, one row, one answer - a query, not a judgement*
MACHINERY	**Find near-duplicate stories**	*The search is machinery. Only the near-miss needs a judgement*
AGENT	**Write the clarifying comment**	*Language over a mess of findings*
HUMAN	**Move the story to Ready**	*A consequence lands on somebody else's sprint*

Three of six. That is a normal answer, and it is the one people find disappointing.

Machinery, agent, human

Three questions, asked of every step, before anybody opens a build tool.

1

MACHINERY

Can you write the rule down?

Defined input, defined output, and a rule that holds. Then write the rule. It is faster, it costs nothing per run, it gives the same answer twice, and you can put a test around it.

A query is not a judgement.

2

AGENT

Is the input messy and the output a judgement?

No rule survives contact with the input, and the output is an opinion somebody could reasonably disagree with. Now you accept the price: a model call, a few seconds, and a different route each run.

This is what you are paying for.

3

HUMAN

Does being wrong cost real money or trust?

The consequence lands on somebody who was not in the room. Keep the step, keep the person, and let the agent do everything up to it - including writing the thing the person approves.

Not a limitation. A choice, made once.

An agent on a step that had a rule is slower, dearer and harder to test. You will be asked to justify all three.

What is behind the word "tool"

Two things wear the same name, and the chat looks identical either way.

WHAT WE BUILT · A LOOKUP

Forty lines of code that read a table and return the row.

- Same input, same output, every time
- You can write a test against it, and it stays green
- Costs nothing per call, and cannot invent an owner

The judgement stays in the agent. The fact comes from the system that owns it.

THE TEMPTING SHORTCUT · A MODEL CALL

Paste the table into the instructions and ask the model to remember it.

- Can be confidently wrong about a fact you own, correctly, in a table
- No exact-match test stays green - two runs need not agree
- The whole table re-billed with every single call

It goes on screen faster, and it is the version most people ship.

In every product, both of these are called a tool. Ask what the tool actually calls.

A rule of thumb that holds up: facts from code, judgement from the model - and never the other way round.

6

Build - the same agent, somewhere else

The terminal: same files, second surface, zero rebuild

LIVE · 20 MINUTES

Build 3 - the setup

Nothing gets rebuilt. The same repository, opened by a different surface.

FIVE FIELDS, SAME FIVE PLACES

Instructions	`.github/agents/story-check.agent.md`
Grounding	`standards/ + .github/instructions/`
Tools	`the tools: list in that same file`
Trigger	`a person - in a terminal this time`
Stop	`an approval prompt, per command`

First the full run in VS Code, end to end. Then the same agent, from the terminal, on a story it has never seen.

FOUR THINGS TO WATCH

1. **The full run writes a file.**
   Verdict, findings, a drafted comment - into reviews/, with the conventions applied.
2. **One deleted line, and the conventions vanish.**
   The applyTo line goes; nothing errors; the output quietly loses its house style.
3. **The stop is a dialog, and one option is standing.**
   Allow once, session, workspace - or always. Watch which one tempts at a Friday pace.
4. **The terminal finds the same agent.**
   No import, no export, no rebuild. It reads the same files, because the agent is the files.

If the agent were the app, this build would be impossible. It is twenty lines of markdown, and that is the finding.

Two surfaces, one agent

What moved between them, and what did not come along.

FIELD	VS CODE CHAT	THE TERMINAL
Instructions	the same agent file	the same agent file
Grounding	the same repository	the same repository
Tools	the picker, plus the frontmatter list	its own tool set - wired separately
Trigger	a person, in the chat box	a person - or a schedule around it
Stop	the dialog, four scopes	an approval prompt, per command

WHAT MOVED

Nothing. The agent is the repository. Anything that reads the repository can run it.

WHAT DID NOT COME ALONG

The tool wiring. Each surface grants its own tool set - check it, never assume it.

Instructions and grounding travel with the repository. Tools and approvals belong to the surface. Split them that way when you design.

Three things to carry out of today

If the rest goes, these three are the ones that pay for the session.

1

Four parts; five fields.

Reasoning, memory, tools, planning is what an agent is. Instructions, grounding, tools, trigger, stop is what you fill in. When it misbehaves, name the part - then fix its field, before anybody blames the model.

2

Grounding fails politely.

A stale copy, a missing applyTo line, a file it never read - none of them shows an error. After every grounding change, ask a question you already know the answer to, and check the reference list on the reply.

3

Judgement is agent work. A rule is machinery.

A lot of disappointing agents are a rule with a model bolted to it - slower than the rule, dearer than the rule, and much harder to test. Mark the steps before you build, and be glad when half of them are machinery.

Two of the three are decisions you make before you open a build tool.

Three things that will stop you on Monday

None of them is a technical limit, and all three cost time if you meet them cold.

PLAN

Most of what you saw needs a paid Copilot plan.

Agent mode, custom agents, MCP and the instructions files sit behind paid tiers, and the free tier's contents move. Check the plans page against what your team actually holds before you design around a feature.

POLICY

An administrator can switch parts of this off, org-wide.

MCP, individual features and specific models are all policy-controlled in business and enterprise plans. If a tool worked at home and not at work, that is usually the reason - ask before you debug.

HOSTING

The unattended version needs the repository on GitHub.

Assigning work to the agent and letting it open pull requests runs on GitHub's infrastructure. Code that lives elsewhere still gets the editor and the terminal - but not the part that works while you sleep.

All three are a procurement conversation, not an engineering one. Have it early.

Before next session

Ten minutes of thinking, and it makes the next session yours rather than mine.

Bring one process from your own work, written out as steps.

Six or eight steps is plenty. One line each. It does not need to be tidy, and the messy ones are more use than the tidy ones.

THEN THREE MARKS AGAINST IT

- Mark every step agent, machinery, or human - using the three questions from earlier
- Note which single step you are least sure about. That one is worth the room’s time
- Note what the process touches: which systems it would have to read from, and write to

Questions

And anything the builds raised that we ran past.

NEXT SESSION

The same agent moves into Microsoft Copilot Studio - and gets a schedule instead of a person